Building the Order Explorer with Claude Code

Conversation transcript for the Frontend Developer Assignment (Problem Statement 1)

Date	9 October 2026
AI tool	Claude Code (VS Code extension), model Claude Opus 5.5
Scope	8 prompts, every Claude reply, and 139 actions Claude took (files written, commands run, checks)
Source	Exported from the local Claude Code session log, unedited apart from the omissions below
Omitted	Claude's internal reasoning, full file contents (see the GitHub repository), system and editor context messages, and internal tool look-ups. Command output is shortened to its first lines.
Redacted	Personal email addresses; local folder paths shortened

PROMPT 1 · 9 OCT, 11:45

Attached file: Frontend Developer Assignment (1).pdf

Hi , I have attached a Frontend Developer Assignment PDF. Please read the complete assignment carefully and help me build it using React.js and TypeScript.

First, analyze the requirements and recommend which problem statement I should choose based on my frontend developer experience. Explain the expected features, recommended tech stack, and project architecture.

Don't start coding yet. First, help me understand the requirements and create a step-by-step implementation plan. We will build and test the project incrementally, ensuring all assignment requirements are met.

CLAUDE

Frontend Developer Assignment: requirements and build plan

1. Which problem to choose

I recommend Problem 1, the data explorer.

	Problem 1: Data Explorer	Problem 2: Offline-safe Form
Backend needed	None. A mock API (MSW) runs in the browser	A real Supabase project, plus storage for the file upload
Main difficulty	Race conditions, URL state, virtualization, accessibility	Offline queue, syncing, conflict resolution, autosave, file upload, validation on both client and server
How easy to test	High. You control the mock API's delay and failures in tests	Lower. Offline sync and two-tab conflicts are hard to test reliably

	Problem 1: Data Explorer	Problem 2: Offline-safe Form
What can go wrong	Low. Everything runs in your own code	Higher. Supabase setup, security rules, storage and offline edge cases
Skills it shows	Core React and TypeScript, data fetching, performance, accessibility	Forms, IndexedDB, sync design

Problem 1 is entirely frontend work, which is what the role is about. Every requirement is something you can show clearly in a 5-minute demo: throttle the network, type fast, watch a stale response get ignored. Problem 2 is a good choice only if you have already built offline-first or sync systems. Otherwise it has many more ways to fail before the deadline.

---

2. Requirements in plain terms (Problem 1)

Dataset and mock API

- At least 10,000 records. I suggest **orders**: id, customer, email, status, total, item count, country, payment method, created date.
- The data is generated from a fixed random seed, so it is the same on every load.
- `GET /api/orders?q=&status=&country=&minTotal=&maxTotal=&sort=createdAt:desc&page=1&pageSize=100` handles search, filters, sorting and pagination on the server side.
- `GET /api/orders/:id` returns one order for the detail view.
- `GET /api/orders/stats?...` returns summary numbers for the current filters. This second request gives us a natural **partial failure**: the rows load but the stats fail, or the other way round.
- Every response waits a random 200 ms to 3 s, and about 10% of requests fail with a 500 error.
- A small "chaos" setting can force failures or fixed delays. You'll use it for the demo and the tests.

The six "must" items, and what each means in practice

1. **Never show stale results**
   - Search waits ~300 ms after the last keystroke before sending a request.
   - Each request is keyed by its exact parameters, and a superseded request is cancelled.
   - A late response from old parameters can never be shown for the new ones.
2. **State in the URL**
   - Search, filters, sort, page and page size all live in the URL, and so does the open detail id.
   - Refresh, back/forward and a pasted link all restore the same view.
   - Invalid URL values fall back safely to defaults.
3. **Fast with large lists**
   - Rows are virtualized, so only the visible rows are rendered, even with 1,000 rows per page.
   - Rows are memoized and callbacks are stable, so nothing re-renders the whole table.
4. **Honest states**
   - First load shows a skeleton.
   - A refetch dims the old rows and shows "Updating…". Old data is never shown as if it were current.
   - Empty results show "No results" with a "Clear filters" button.
   - An error shows a message and a Retry button. Any old data is hidden or clearly marked out of date.

- Partial failure: the rows show, the stats panel shows its own error and Retry.

5. **Accessible**
   - Everything works with the keyboard alone: Tab, arrow keys between rows, Enter to open, Esc to close.
   - Focus is always visible.
   - Sortable headers use `aria-sort`, and pagination uses `aria-current`.
   - A live region announces "1,234 results", and errors are announced with `role="alert"`.
6. **Detail view**
   - The route is `/orders/:id?<same params>`, shown as a drawer on top of the list. The list stays mounted, so its scroll position and filters survive.
   - Closing the drawer returns focus to the row that opened it.
   - Opening the link directly loads the list and the drawer together.

Submission items: a Vercel deployment, a demo video under 5 minutes, a public GitHub repo with a README (architecture, tradeoffs, sources), tests, and a note on AI use. You can share this conversation as your AI-usage export.

---

3. Tech stack

Concern	Choice	Why
Build	**Vite + React 18 + TypeScript (strict)**	Fast and standard
Routing / URL	**React Router v6**	Nested route for the drawer; search-param helpers
URL parsing	**Zod**	Typed parse and serialize, with safe defaults for bad values
Server state	**TanStack Query v5**	Caches by parameters, cancels via `AbortSignal`, `placeholderData` for refetches, retry
Virtualization	**TanStack Virtual**	Headless, so we keep full control of accessibility markup
Mock API	**MSW v2** + `@faker-js/faker` (seeded)	Same handlers in the browser, the deployed app and the tests
Drawer / dialog	**Radix Dialog** (or native `<dialog>`)	Focus trap and focus return built in
Styling	**Tailwind CSS** (or CSS Modules)	Quick, with easy `focus-visible` styles
Unit / integration tests	**Vitest + Testing Library + MSW (node)**	Race-condition and state tests with controlled delays
End-to-end tests	**Playwright** + `@axe-core/playwright`	URL restore, back/forward, keyboard flow, accessibility scan
Deploy	**Vercel**	SPA rewrite so deep links work

I deliberately left out Redux or Zustand. Server state lives in TanStack Query and view state lives in the URL, so there is no separate global store to keep in sync. That is a strong architecture point for the README.

---

4. Architecture

```
src/
  mocks/
    data.ts            # seeded generator → 10k orders
    query.ts           # pure fn: filter/search/sort/paginate (unit-tested)
    chaos.ts           # latency + failure rate, overridable
    handlers.ts        # MSW routes using query.ts + chaos.ts
    browser.ts / server.ts
  api/
    client.ts          # fetch wrapper, AbortSignal, typed errors
    orders.ts          # fetchOrders, fetchOrder, fetchStats
  features/explorer/
    params.ts          # Zod schema: URL ⇄ ExplorerParams
    useExplorerParams.ts
    useOrdersQuery.ts  # TanStack Query hooks
    components/
      SearchBox.tsx    # local input state + debounce → URL
      Filters.tsx
      OrdersTable.tsx  # virtualized, sortable headers
      OrderRow.tsx     # React.memo
      Pagination.tsx
      ResultStatus.tsx # live region: counts, updating, errors
      StatsPanel.tsx   # independent query → partial failure
      states/ (Skeleton, Empty, ErrorState)
  features/detail/
    OrderDrawer.tsx    # route /orders/:id, focus return
  hooks/useDebouncedValue.ts
  App.tsx / main.tsx   # router, QueryClient, MSW bootstrap
tests/ (e2e Playwright)
```

How data flows: URL → `useExplorerParams` (Zod-parsed) → query key → TanStack Query → table.

- UI changes write to the URL and never to local copies. Typing uses `replace` so each keystroke doesn't add a history entry. Discrete changes such as a filter, sort or page use `push`, so Back undoes them.
- When the user presses Back or Forward, the search box re-syncs from the URL.

---

5. Step-by-step plan

We finish and test each phase before moving to the next.

#	Phase	What we build	How we verify it
0	**Scaffold**	Vite React TS, ESLint/Prettier, Tailwind, Vitest, Playwright, git repo	`npm run dev`, an empty test passes
1	**Mock API**	Seeded 10k dataset, query logic, MSW handlers, chaos settings	Unit tests on `query.ts`; delays and failures visible in the Network tab
2	**URL state**	Zod schema, `useExplorerParams`, push vs replace rules	Tests: round-trip, invalid values, defaults
3	**Data layer**	API client with abort, query hooks, debounce	**Race test**: a slow request for "a" resolves after a fast one for "ab", and only "ab" results show
4	**Table**	Virtualized table, sortable headers, memoized rows	Smooth at 1,000 rows/page; React Profiler shows no wasted renders

#	Phase	What we build	How we verify it
5	**Controls**	Search, filters, pagination, page size	Back/forward and refresh restore the view (Playwright)
6	**Honest states**	Skeleton, updating, empty, error + retry, stale marking, stats partial failure	Tests force a 500 → error and Retry shown, old rows not shown as current
7	**Detail drawer**	`/orders/:id` nested route, deep link, focus return, scroll kept	E2E: open → close → scroll position and filters unchanged
8	**Accessibility pass**	Roving focus in the table, live regions, `aria-sort`, focus styles	Keyboard-only walkthrough, axe scan, quick NVDA check
9	**Deploy**	MSW enabled in production, Vercel SPA rewrites	The live URL works and deep links load
10	**Docs and demo**	README (setup, architecture, tradeoffs, sources, AI use), demo script	Checklist against the PDF

Tradeoffs to note in the README:

- **Pagination vs infinite scroll.** I chose numbered pages plus virtualization because a page number restores exactly from the URL. Infinite scroll would need scroll offsets stored in the URL.
- **MSW instead of a real backend.** It runs in the deployed app, and the tests are deterministic.
- **No global store.** Server state lives in TanStack Query and view state lives in the URL.

---

6. Before we start Phase 0

1. Are you happy with **Problem 1** and the **orders** dataset? Support tickets or products would work just as well.
2. Do you prefer **Tailwind** or plain **CSS Modules**?
3. I'll set the project up in `d:\frontend-web` unless you'd like another folder.

Once you confirm, I'll start with Phase 0 and Phase 1 and we'll test them before going further.

PROMPT 2 · 9 OCT, 11:49

yes we can start

CLAUDE'S ACTIONS (1)

▸ Check project folder and tool versions

```
$ ls -la && node -v && npm -v && git --version
```

```
total 8
drwxr-xr-x 1 team 197609 0 Oct  9 11:48 .
drwxr-xr-x 1 team 197609 0 Oct  9 11:38 ..
v24.13.1
11.8.0
git version 2.53.0.windows.1
```

CLAUDE

The folder is empty and Node 24 is available. I'll scaffold the project by hand (rather than an interactive generator) and install current stable packages.

CLAUDE'S ACTIONS (1)

▸ Init git and install dependencies

```
$ git init -q && cat > package.json <<'EOF'
```

```
  run `npm fund` for details
found 0 vulnerabilities
  run `npm fund` for details
found 0 vulnerabilities
order-explorer@0.1.0 D:\frontend-web
+-- @eslint/js@10.0.1
+-- @faker-js/faker@10.6.0
+-- @tailwindcss/vite@4.3.3
+-- @tanstack/react-query@5.104.1
+-- @tanstack/react-virtual@3.14.13
+-- @testing-library/jest-dom@7.0.1
+-- @testing-library/react@16.3.3
+-- @testing-library/user-event@14.6.7
+-- @types/node@26.6.4
… (19 more lines)
```

CLAUDE

Installed versions are newer than in my plan (React 19, React Router 8, Vite 8, Vitest 5, TypeScript 6, Zod 4). Before writing code against them, I'll check the APIs I'll rely on.

CLAUDE'S ACTIONS (3)

▸ Inspect installed package versions and exports

```
$ cd ./node_modules && for p in react-router vite vitest typescript zod msw; do echo "== $p"; node -e
"const p=require('./$p/package.json');console.log(p.version, JSON.stringify(Object.keys(p.exports||{ …
```

```
== react-router
8.4.0 [".","./dom","./route-pattern","./internal","./internal/react-server-client","./package.json"]
{"node":">=22.22.0"}
== vite
8.3.4 [".","./client","./module-
runner","./internal","./dist/client/*","./types/*","./types/internal/*","./package.json"]
{"node":"^20.19.0 || >=22.12.0"}
== vitest
5.0.3 [".","./browser","./package.json","./optional-types.js","./optional-runtime-
types.js","./globals","./jsdom","./importMeta","./import-meta","./node","./internal/brow …
== typescript
6.0.3 [] {"node":">=14.17"}
== zod
4.6.5 ["./package.json",".","./mini","./compile","./locales","./v3","./v4","./v4-
mini","./v4/mini","./v4/core","./v4/locales","./v4/locales/*"]
== msw
2.15.0
[".","./browser","./node","./native","./core/http","./core/graphql","./core/ws","./experimental","./moc
kServiceWorker.js","./package.json"] {"node":">=18"}
BrowserRouter:function createBrowserRouter:function RouterProvider:function useSearchParams:function
Routes:function Route:function Outlet:function useNavigate:function u …
development
… (15 more lines)
```

▸ Read React Router v8 breaking changes

```
$ cd . && awk '/^## v8.0.0/,/^## v7/' node_modules/react-router/CHANGELOG.md | grep -E '^- ' | head
-30; echo; grep -n "^## " node_modules/vitest/package.json >/dev/null; ls node_modules/vitest/dist | h
…
```

```
- Remove the `future.v8_trailingSlashAwareDataRequests` flag ([#15100](https://github.com/remix-
run/react-router/pull/15100))
- Update `tsconfig.json` `target`/`lib` from `ES2020 -> ES2022` ([591853e](https://github.com/remix-
run/react-router/commit/591853e))
- Switch the published packages in `packages/` to ESM-only. ([#14895](https://github.com/remix-
run/react-router/pull/14895)) ([59ebcf1](https://github.com/remix-run/react …
- Remove deprecated `data` parameter in favor of `loaderData` for `meta` APIs (to align with
`Route.ComponentProps`) ([#14931](https://github.com/remix-run/react-router/p …
- Remove `future.v8_passThroughRequests` flag - the raw incoming `request` is now always passed through
to `loader`/`action`. Use `url` for the normalized URL without Rea …
- Remove internal `hasErrorBoundary` field added to `router.routes` when using a data router ([#15074]
(https://github.com/remix-run/react-router/pull/15074))
- Remove `react-router-dom` package ([#15076](https://github.com/remix-run/react-router/pull/15076))
- Remove `future.v8_middleware` flag — middleware is always enabled in v8 ([#15078]
(https://github.com/remix-run/react-router/pull/15078))
- Update minimum Node version to 22.22.0 ([#14928](https://github.com/remix-run/react-
router/pull/14928))
- Update minimum React version to 19.2.7 ([#15062](https://github.com/remix-run/react-
router/pull/15062))
- Bump dependencies ([#15080](https://github.com/remix-run/react-router/pull/15080))
- Ensure client middleware errors load lazy route error boundaries before bubbling ([#15086]
(https://github.com/remix-run/react-router/pull/15086))
- Remove explicit `onSubmit` type override from `SharedFormProps` to fix deprecation warning with
`@types/react@19.x` ([#14932](https://github.com/remix-run/react-router/ …
- Update package builds to preserve individual module files in published artifacts. Public APIs and
documented import paths are unchanged. ([#15092](https://github.com/re …
… (8 more lines)
```

▸ Write Phase 0 config files and init MSW worker

```
$ npm uninstall @faker-js/faker 2>&1 | tail -1 && mkdir -p src/shared src/mocks src/test public && npx
msw init public --save 2>&1 | tail -2
```

```
found 0 vulnerabilities
Updating "msw.workerDirectory" at "package.json"...
39:    "msw": "^2.15.0",
47:  "msw": {
```

CLAUDE

Now Phase 1: the shared types and query schema. The client and the mock server both use them, so the URL and API formats can't drift apart.

CLAUDE'S ACTIONS (3)

- ▸ Wrote file `src\shared\order.ts` (78 lines)
- ▸ Wrote file `src\shared\orderQuery.ts` (95 lines)
- ▸ Wrote file `src\mocks\data.ts` (170 lines)

CLAUDE

Next, the pure server-side query logic, the chaos (latency and failure) layer, and the MSW handlers:

CLAUDE'S ACTIONS (4)

- Wrote file `src\mocks\query.ts` (97 lines)
- Wrote file `src\mocks\chaos.ts` (65 lines)
- Wrote file `src\mocks\handlers.ts` (48 lines)
- Write MSW setup, entry point and placeholder App

```
$ cat > src/mocks/browser.ts <<'EOF'
```

```
ok
```

CLAUDE

Now the Phase 1 tests: the URL schema, the dataset, the query logic, and the handlers end to end through MSW's Node server.

CLAUDE'S ACTIONS (1)

- Write Phase 1 tests and run them

```
$ cat > src/shared/orderQuery.test.ts <<'EOF'
```

```
Exit code 2
/usr/bin/bash: -c: line 188: unexpected EOF while looking for matching `''
```

CLAUDE

The shell rejected that heredoc, so I'll write the test files with the Write tool.

CLAUDE'S ACTIONS (6)

- Check which test files were written

```
$ ls src/shared src/mocks
```

```
src/mocks:
browser.ts
chaos.ts
data.ts
handlers.ts
query.ts
server.ts
src/shared:
order.ts
orderQuery.ts
```

- Wrote file `src\shared\orderQuery.test.ts` (67 lines)
- Wrote file `src\mocks\data.test.ts` (26 lines)
- Wrote file `src\mocks\query.test.ts` (88 lines)
- Wrote file `src\mocks\handlers.test.ts` (67 lines)
- Run the test suite

```
$ npx vitest run 2>&1 | tail -40
```

```
 RUN  v5.0.3 D:/frontend-web
 Test Files  4 passed (4)
      Tests  22 passed (22)
   Start at  11:55:39
   Duration  5.65s (environment 54%, tests 23%, setup 9%, import 8%, transform 5%)
```

CLAUDE

All 22 tests pass. Next, the type check, lint and production build:

CLAUDE'S ACTIONS (1)

- Typecheck, lint and build

```
$ npx tsc --noEmit 2>&1 | tail -20; echo "tsc exit $?"; npx eslint . 2>&1 | tail -20; echo "eslint
done"; npx vite build 2>&1 | tail -12
```

```
tsc exit 0
src\mocks\query.ts
  63:14  error  '_lineItems' is defined but never used        @typescript-eslint/no-unused-vars
  64:20  error  '_shippingAddress' is defined but never used  @typescript-eslint/no-unused-vars
  65:18  error  '_shippingCents' is defined but never used    @typescript-eslint/no-unused-vars
✖ 3 problems (3 errors, 0 warnings)
eslint done
✓ built in 3.23s
[plugin builtin:vite-reporter]
(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rolldownOptions.output.codeSplitting to improve chunking:
https://rolldown.rs/reference/OutputOptions.codeSplitting
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
[PLUGIN_TIMINGS] JavaScript callbacks ran for 3.0s of this 3.2s build (93%).
… (4 more lines)
```

CLAUDE

Two things to fix: the lint rule should allow underscore-prefixed unused variables (the usual convention), and one chunk is over 500 kB. I'll see which one.

CLAUDE'S ACTIONS (7)

▸ Allow underscore unused vars and list bundle sizes

```
$ sed -i "s|      'react-refresh/only-export-components': \['warn', { allowConstantExport: true
}\],|&\n      '@typescript-eslint/no-unused-vars': [\n        'error',\n        { argsIgnorePattern:
'^_', …
```

```
LINT_OK
0 .
0 ..
512362 browser-2Vj8DzCy.js
5315 index-BFuYfFXz.css
222005 index-DJVHO8qp.js
```

▸ Edited file `vite.config.ts`

▸ Install Playwright and its bundled Chromium

```
$ npm install -D @playwright/test 2>&1 | tail -1 && npx playwright install chromium 2>&1 | tail -3
```

```
found 0 vulnerabilities
|■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■        |  90% of 118.1 MiB
|■■■■■■■■■■■■■■■■■■■
```

▸ Run typecheck, lint, unit tests and show git status

```
$ npm run typecheck 2>&1 | tail -3 && npm run lint 2>&1 | tail -3 && npm test 2>&1 | grep -E "Test
Files|Tests " ; git status --short | head -30
```

```
> order-explorer@0.1.0 typecheck
> tsc --noEmit
> order-explorer@0.1.0 lint
> eslint .
 Test Files  4 passed (4)
      Tests  22 passed (22)
?? .gitignore
?? .prettierrc
?? e2e/
?? eslint.config.js
?? index.html
?? package-lock.json
?? package.json
?? playwright.config.ts
… (4 more lines)
```

CLAUDE

Phases 0 and 1 are done. The project is set up and the mock API works, with its random latency and failures. All checks pass: 22 unit tests, 3 browser tests against the production build, the type check, lint and the build.

What's in place

- **Project setup:** Vite, React 19, TypeScript in strict mode, Tailwind, ESLint, Prettier, Vitest and Playwright. Package versions are newer than in my plan (React Router 8, Vite 8, Zod 4); the APIs we need are unchanged.
- **Query format:** orderQuery.ts defines the parameters (search, filters, sort, page) once. The mock API and, from Phase 2, the URL will both use it.
  - A bad value in a link, like `page=-4`, falls back to its default instead of breaking the page.
  - The same view always produces the same URL.
- **Dataset:** data.ts generates 12,000 orders from a fixed seed, so you get identical data on every load and on every machine. As mentioned above, I wrote a small generator instead of using Faker, which would have added several MB to the deployed app.
- **Mock API:** search, filtering, sorting and pagination in query.ts, served by handlers.ts:
  - `GET /api/orders` returns a page of results.
  - `GET /api/orders/stats` returns summary numbers. We'll use it for the partial-failure state.
  - `GET /api/orders/:id` returns one order, or a 404 if it doesn't exist.
  - Search ignores case and accents, so "muller" finds "Müller".
- **Simulated network:** chaos.ts adds a random 200 ms–3 s delay and fails about 1 in 10 requests by default. Tests can override this, and so can you for the demo via localStorage.
- **Mock API in production:** MSW runs as a service worker on the live site too, because it is the app's backend. A browser test confirms that requests are answered by the service worker in the production build.

Try it: run `npm run dev`, open the page and refresh a few times. The page is still a temporary placeholder, so you'll see the delay vary and sometimes an HTTP 500/503. The Network tab shows the

requests. To force failures, run this in the console and refresh:

```
localStorage.setItem('order-explorer:chaos', JSON.stringify({ failureRate: 1 }))
```

Remove the key to go back to the defaults.

Git: the repo is initialized but nothing is committed yet. Should I make the first commit?

Next is Phase 2, the URL state hook: it reads and writes the URL, uses `replace` while you type and `push` for filter, sort and page changes, and stays in sync with Back/Forward. After that comes Phase 3, the data layer, with the test that a slow, outdated response never overwrites a newer one. Shall I continue?

PROMPT 3 · 9 OCT, 12:29

First, fix the mock API issue I reported. The UI shows 12,000 orders, but only 2–4 records are appearing in the actual response.

Please inspect the existing code, generate and persist exactly 12,000 unique orders, and verify that server-side pagination returns 25 different records per page across 480 pages. Confirm the actual dataset size instead of hardcoding the count.

Once this is fixed and verified, continue building the complete React.js application according to my original assignment PDF.

Important:

- Follow every requirement in the original assignment.
- Reuse the existing working code instead of rebuilding everything unnecessarily.
- Implement all required features, including search, debouncing, filtering, sorting, pagination, URL state persistence, race-condition handling, error states, retries, and order details.
- Use a clean, professional, responsive UI.
- Write and run relevant automated tests.
- Verify each feature before marking it complete.
- Do not skip requirements or use fake functionality.
- Explain which files you changed and how I can run and test the application locally.

Start by fixing and verifying the data issue. Then proceed step by step until the entire assignment is complete.

CLAUDE

I'll start by finding where the "2–4 records" comes from. The temporary page in App.tsx, which is still the Phase 1 placeholder, has this line:

```
JSON.stringify(page.items.slice(0, 3), null, 2)
```

It only *displays* the first 3 items of each response. I'll confirm in a real browser what the API actually returns, rather than assume.

CLAUDE'S ACTIONS (7)

▸ Wrote file `e2e\dataset.spec.ts` (60 lines)

▸ Verify dataset and pagination through the service worker

```
$ npx playwright test e2e/dataset.spec.ts 2>&1 | tail -15
```

```
        at e2e\dataset.spec.ts:56:24
    Error Context: test-results\dataset-the-dataset-is-identical-after-a-reload-chromium\error-
context.md
    attachment #2: trace (application/zip) ─────────────────────────────────────────────────────
    test-results\dataset-the-dataset-is-identical-after-a-reload-chromium\trace.zip
    Usage:
        npx playwright show-trace test-results\dataset-the-dataset-is-identical-after-a-reload-
chromium\trace.zip
    ─────────────────────────────────────────────────────────────────────────────────────────────

  2 failed
    [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the whole
dataset once
    [chromium] › e2e\dataset.spec.ts:48:1 › the dataset is identical after a reload ──────────────
```

▸ Show the dataset test failure details

```
$ npx playwright test e2e/dataset.spec.ts 2>&1 | grep -v "^\s*$" | head -40
```

```
Running 2 tests using 2 workers
  x  1 [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the
whole dataset once (423ms)
  x  2 [chromium] › e2e\dataset.spec.ts:48:1 › the dataset is identical after a reload (381ms)
  1) [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the whole
dataset once
    Error: page.evaluate: SyntaxError: Unexpected token '<', "<!doctype "... is not valid JSON
      19 |
      20 | test('every page of 25 is full and distinct, covering the whole dataset once', async ({ page
}) => {
    > 21 |   const result = await page.evaluate(async () => {
         |                              ^
      22 |     const get = async (p: number) =>
      23 |       (await (await fetch(`/api/orders?pageSize=25&page=${p}`)).json()) as PageBody;
      24 |     const first = await get(1);
        at e2e\dataset.spec.ts:21:29
    Error Context: test-results\dataset-every-page-of-25-i-bc52f-ring-the-whole-dataset-once-
chromium\error-context.md
… (25 more lines)
```

▸ Add readiness marker and rerun dataset verification

```
$ cat > src/main.tsx <<'EOF'
```

```
2:import { gotoReady, NO_CHAOS, setChaos } from './fixtures';
17:  await gotoReady(page);
48:test('the dataset is identical after a reload', async ({ page }) => {
57:  await page.reload();
58:  await page.locator('html[data-mock-api="ready"]').waitFor({ state: 'attached' });
  ok 1 [chromium] › e2e\dataset.spec.ts:48:1 › the dataset is identical after a reload (832ms)
Dataset check: {"total":12000,"pageCount":480,"pagesFetched":480,"itemsPerPage":
[25],"idCount":12000,"uniqueIds":12000,"firstIdsOfPage1":["ORD-112000","ORD-111999","ORD-1 …
  ok 2 [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the
whole dataset once (2.8s)
  2 passed (7.9s)
```

▸ Probe jsdom feature support

```
$ cat > src/test/probe.test.ts <<'EOF'
```

```
 Test Files  1 passed (1)
      Tests  1 passed (1)
```

▸ Probe jsdom feature support

```
$ cat > src/test/probe.test.ts <<'EOF'
```

```
Error: showModal:undefined RO:undefined scrollTo:undefined
```

▸ Find measurement APIs used by TanStack Virtual

```
$ cd ./node_modules/@tanstack/virtual-core/dist/esm && grep -n
"getBoundingClientRect\|offsetWidth\|offsetHeight\|ResizeObserver\|scrollTop\|scrollTo(" index.js |
head -30
```

```
15:  const { offsetWidth, offsetHeight } = element;
16:  return { width: offsetWidth, height: offsetHeight };
43:  if (!targetWindow.ResizeObserver) {
47:  const observer = new targetWindow.ResizeObserver((entries) => {
59:    instance.options.useAnimationFrameWithResizeObserver ? requestAnimationFrame(run) : run();
121:  return horizontal ? el.scrollLeft * (isRtl && -1 || 1) : el.scrollTop;
151:  return element[instance.options.horizontal ? "offsetWidth" : "offsetHeight"];
224:        if (!this.targetWindow || !this.targetWindow.ResizeObserver) {
227:        return _ro = new this.targetWindow.ResizeObserver((entries) => {
250:            this.options.useAnimationFrameWithResizeObserver ? requestAnimationFrame(run) : run();
300:        useAnimationFrameWithResizeObserver: false,
929:          // changes size *below* the anchor point, so shifting scrollTop by the
1187:  // Returns `true` when it performed a synchronous `scrollTop` write this
```

CLAUDE

Now building the API client and React Query hooks, the base for the explorer UI.

CLAUDE'S ACTIONS (1)

▸ Read TanStack Virtual element-rect observer

```
$ sed -n 10,60p index.js
```

```
};
const _resetIOSDetectionForTests = () => {
  _isIOSResult = void 0;
};
const getRect = (element) => {
  const { offsetWidth, offsetHeight } = element;
  return { width: offsetWidth, height: offsetHeight };
};
const defaultKeyExtractor = (index) => index;
const defaultRangeExtractor = (range) => {
  const start = Math.max(range.startIndex - range.overscan, 0);
  const end = Math.min(range.endIndex + range.overscan, range.count - 1);
  const len = end - start + 1;
  const arr = new Array(len);
… (37 more lines)
```

CLAUDE

The tests will use small stubs: element size, `scrollTo`, and the dialog API. With those in place, I'm writing the application layer. First the API client, formatting helpers and URL-state hook:

CLAUDE'S ACTIONS (7)

- ▸ Wrote file `src\api\client.ts` (52 lines)
- ▸ Wrote file `src\api\orders.ts` (65 lines)
- ▸ Wrote file `src\queryClient.ts` (21 lines)
- ▸ Wrote file `src\lib\format.ts` (34 lines)
- ▸ Wrote file `src\features\explorer\useOrderQueryParams.ts` (65 lines)
- ▸ Wrote file `src\hooks\useDebouncedDraft.ts` (47 lines)
- ▸ Fix normalize identity and echo tracking in draft hook

```
$ cd . && python - <<'PY'
```

```
3:const identity = (v: string) => v;
19:    normalize = identity,
35:    setLastOwnCommit(null); // only the very next change can be the echo of our own commit
```

CLAUDE

Now the UI components. I'll start with the shared pieces (status badge, error and empty states), then the explorer's search, filters and stats:

CLAUDE'S ACTIONS (5)

- ▸ Wrote file `src\components\ui.tsx` (102 lines)

- Wrote file `src\features\explorer\SearchBox.tsx` (64 lines)
- Wrote file `src\features\explorer\FiltersPanel.tsx` (201 lines)
- Wrote file `src\features\explorer\ActiveFilters.tsx` (65 lines)
- Wrote file `src\features\explorer\StatsPanel.tsx` (74 lines)

CLAUDE

Now the core piece: the virtualized, keyboard-navigable grid.

CLAUDE'S ACTIONS (3)

- Wrote file `src\features\explorer\OrdersGrid.tsx` (345 lines)
- Wrote file `src\features\explorer\Pagination.tsx` (101 lines)
- Fix label visibility, loading text and redundant callback

```
$ python - <<'PY'
```

```
done
```

CLAUDE

Now the page that ties it together, the detail drawer, and the network-simulation control. Reviewers can use that control on the live site to force failures or slow responses.

CLAUDE'S ACTIONS (6)

- Wrote file `src\features\explorer\ExplorerPage.tsx` (242 lines)
- Wrote file `src\features\detail\OrderDrawer.tsx` (202 lines)
- Wrote file `src\features\simulation\NetworkSimulator.tsx` (99 lines)
- Wrote file `src\App.tsx` (65 lines)

▸ Write global CSS and typecheck plus lint

```
$ cat > src/index.css <<'EOF'
```

```
src/features/detail/OrderDrawer.tsx(84,43): error TS2339: Property 'isNotFound' does not exist on type
'Error'.
src\components\ui.tsx
  14:14  warning  Fast refresh only works when a file only exports components. Use a new file to share
constants or functions between components  react-refresh/only-expor …
  42:14  warning  Fast refresh only works when a file only exports components. Use a new file to share
constants or functions between components  react-refresh/only-expor …
src\features\explorer\ActiveFilters.tsx
  28:17  warning  Fast refresh only works when a file only exports components. Use a new file to share
constants or functions between components  react-refresh/only-expor …
src\features\explorer\OrdersGrid.tsx
  98:23  warning  Compilation Skipped: Use of incompatible library
This API returns functions which cannot be memoized without leading to stale UI. To prevent this, by
default React Compiler will skip memoizing this component/hook. Howev …
src\features\explorer\OrdersGrid.tsx:98:23
   96 |   );
   97 |
>  98 |   const virtualizer = useVirtualizer({
      |                       ^^^^^^^^^^^^^^ TanStack Virtual's `useVirtualizer()` API returns
functions that cannot be memoized safely
… (6 more lines)
```

▸ Type query errors, split constants, fix lint

```
$ python - <<'PY'
```

```
(Bash completed with no output)
```

CLAUDE

The type check and lint are clean. I'll format the code, start the production preview, and look at the UI in Playwright's isolated Chromium:

CLAUDE'S ACTIONS (2)

▸ Format code and build production bundle

```
$ npx prettier --write "src/**/*.{ts,tsx,css}" "e2e/**/*.ts" > /dev/null && npm run build 2>&1 | grep -
E "error|built|kB" | head;
```

```
dist/index.html                   0.39 kB │ gzip:   0.26 kB
dist/assets/index-B3tkBuvN.css    26.70 kB │ gzip:   6.18 kB
dist/assets/browser-D-1ypn5I.js  427.57 kB │ gzip: 161.10 kB
dist/assets/index-D5xtj08d.js    440.16 kB │ gzip: 134.51 kB
✓ built in 364ms
[This command modified 16 files you've previously read: src\features\explorer\ExplorerPage.tsx,
src\App.tsx, src\components\ui.tsx, src\mocks\handlers.ts, src\features\ex …
```

▸ Serve production build on port 4173

```
$ npx vite preview --port 4173 --strictPort
```

```
Command running in background with ID: bl0mtdo1l. Output is being written to:
~\AppData\Local\Temp\claude\d--frontend-web\65358903-840e-4a17-864a-55ca4f2ff87d\tasks\bl0mt …
```

CLAUDE

Prettier's output shows the accent-stripping regex as literal combining characters. I'll check whether it actually rewrote the `\u0300-\u036f` escapes:

CLAUDE'S ACTIONS (3)

▸ Inspect regex escapes after formatting

```
$ grep -n "normalize('NFD')" -A1 src/mocks/query.ts src/mocks/data.ts | cat -v | head
```

```
src/mocks/query.ts:13:export const normalizeText = (s: string) => s.normalize('NFD').replace(/[M-LM-^@-
M-MM-/]/g, '').toLowerCase();
src/mocks/query.ts-14-
--
src/mocks/data.ts:213:        .normalize('NFD')
src/mocks/data.ts-214-        .replace(/[M-LM-^@-M-MM-/]/g, '')
```

▸ Replace raw combining chars with Unicode property escape

```
$ python - <<'PY'
```

```
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
  File "C:\Python314\Lib\re\__init__.py", line 208, in sub
    return _compile(pattern, flags).sub(repl, string, count)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^
  File "C:\Python314\Lib\re\__init__.py", line 377, in _compile_template
    return _sre.template(pattern, _parser.parse_template(repl, pattern))
                                  ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^
  File "C:\Python314\Lib\re\_parser.py", line 1061, in parse_template
    raise s.error('bad escape %s' % this, len(this)) from None
re.PatternError: bad escape \p at position 1
```

▸ Replace raw combining chars with Unicode property escape

```
$ python - <<'PY'
```

```
<stdin>:5: SyntaxWarning: "\p" is an invalid escape sequence. Such sequences will not work in the
future. Did you mean "\\p"? A raw string is also an option.
src/mocks/data.ts:214:        .replace(/\p{M}/gu, '')
src/mocks/query.ts:13:export const normalizeText = (s: string) => s.normalize('NFD').replace(/\p{M}/gu,
'').toLowerCase();
      Tests  17 passed (17)
```

CLAUDE

> Accent-insensitive search still passes. Next, screenshots of the running app at desktop and mobile widths:

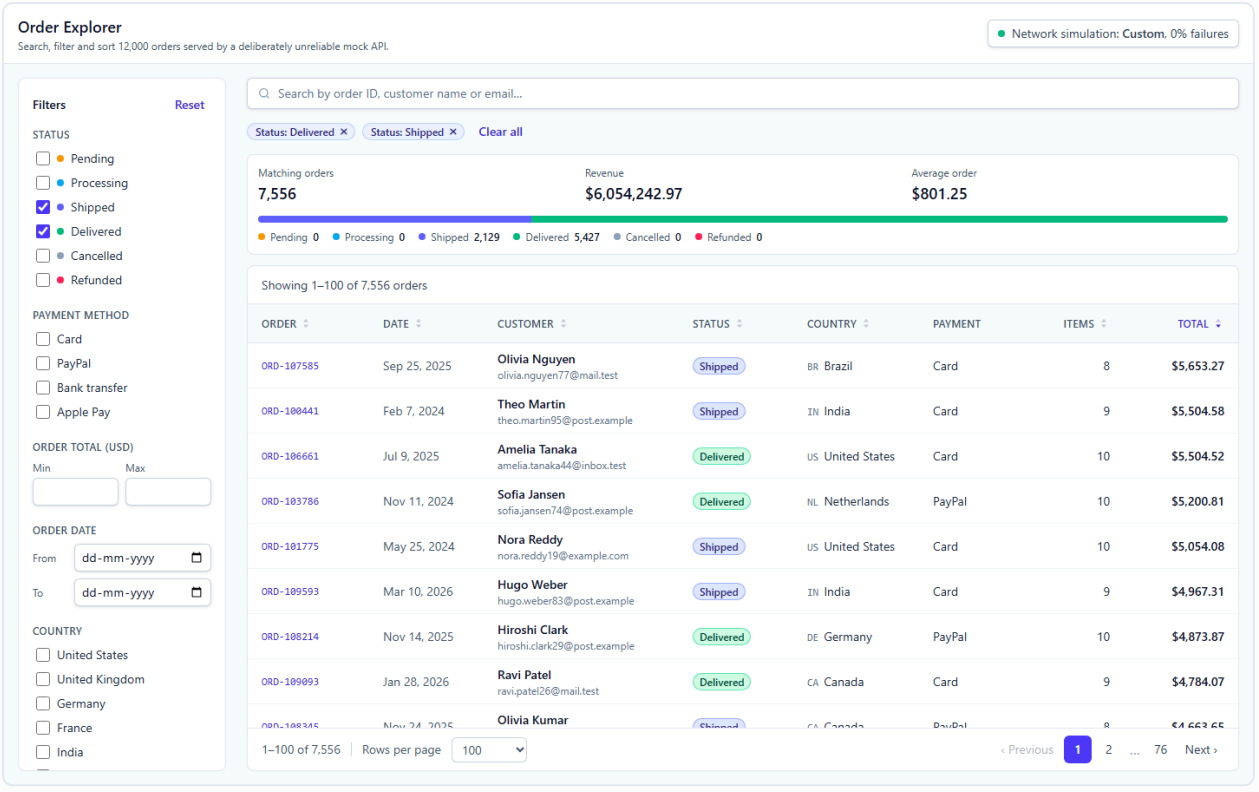
CLAUDE'S ACTIONS (6)

▸ Capture desktop, drawer and mobile screenshots

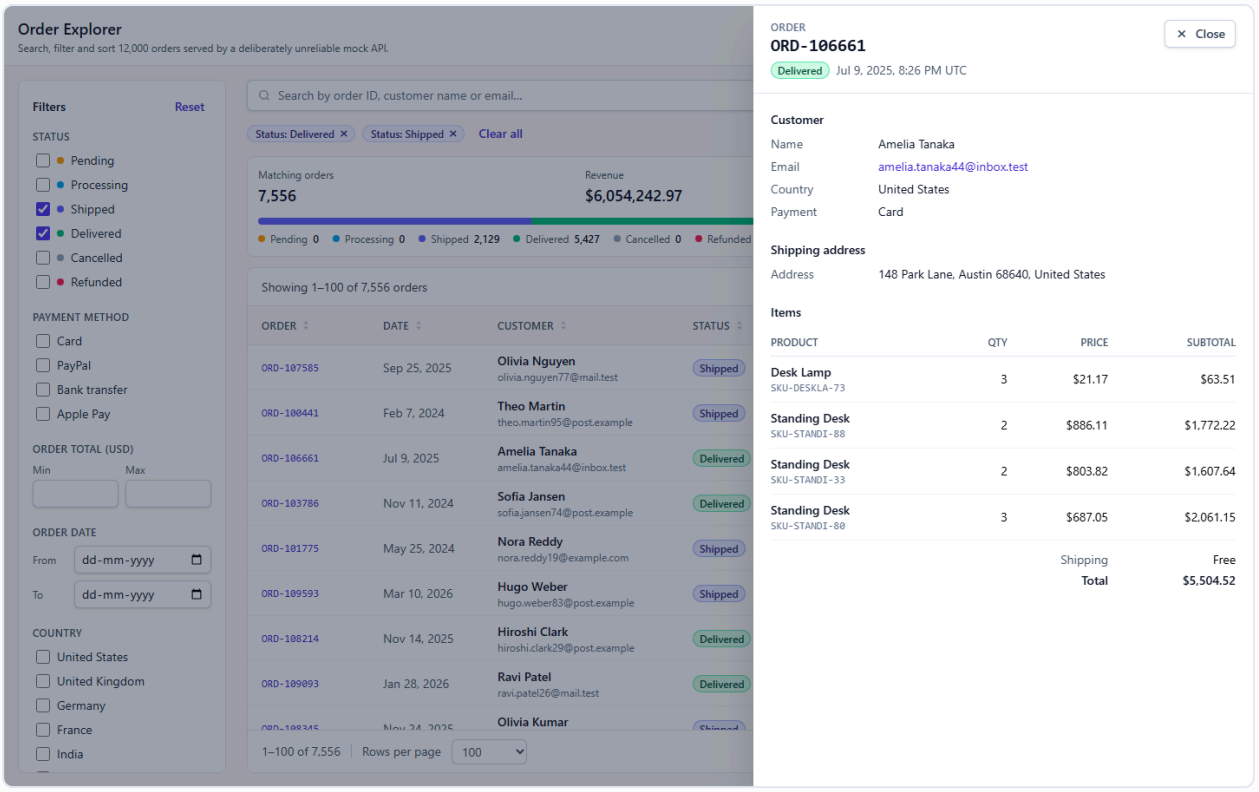
```
$ SP="<scratchpad>"; cat > "$SP/shot.cjs" <<'EOF'
```

```
no console errors
```

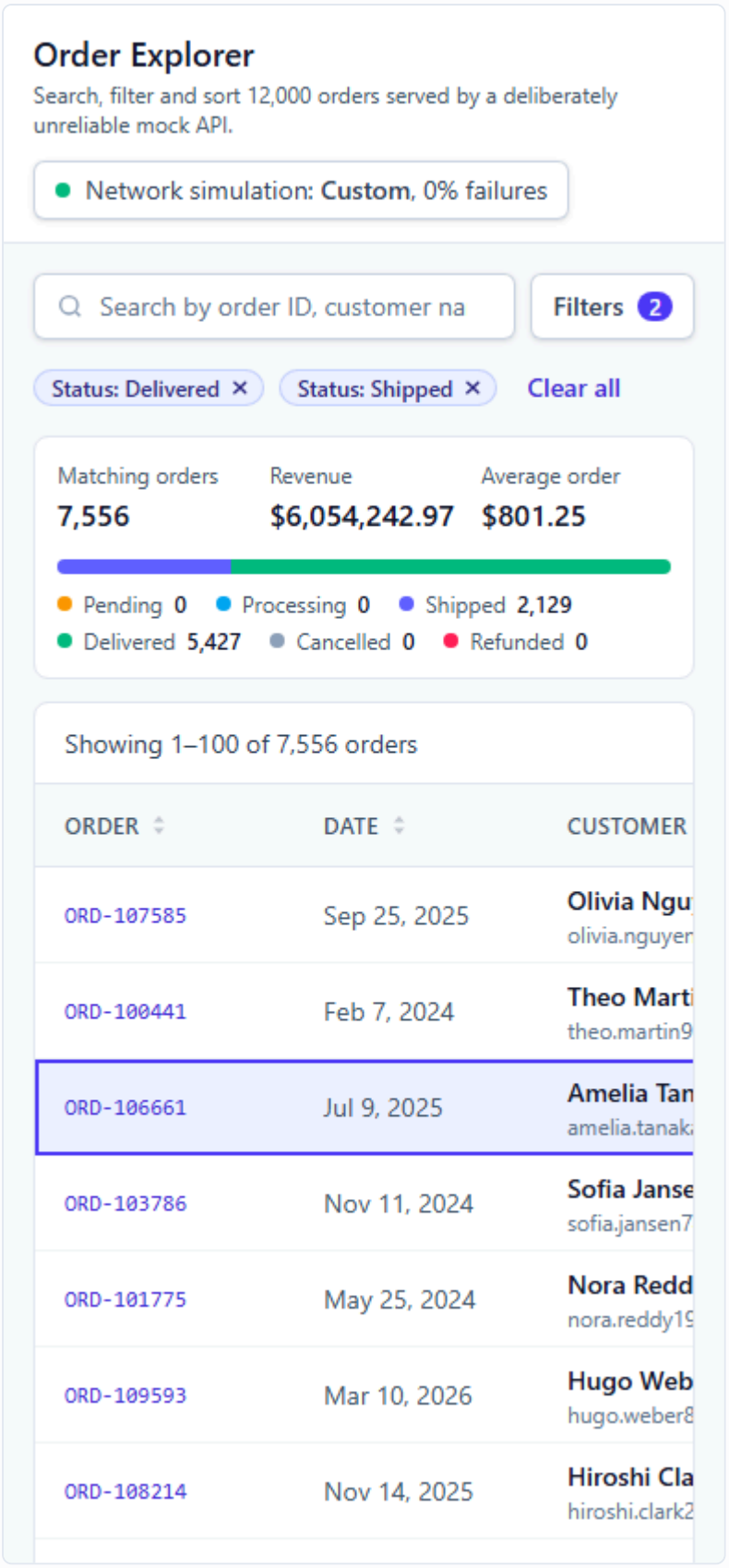
▸ Read `<scratchpad>\desktop.png`



▸ Read `<scratchpad>\drawer.png`



▸ Read `<scratchpad>\mobile.png`



▸ Wrote file `src\test\setup.ts` (36 lines)

▸ Move MSW lifecycle into setup and rerun tests

```
$ python - <<'PY'
```

```
⎯⎯⎯⎯⎯⎯⎯ Failed Tests 1 ⎯⎯⎯⎯⎯⎯⎯
 FAIL  src/mocks/handlers.test.ts > default chaos > fails roughly 1 in 10 requests
AssertionError: expected 0 to be greater than 10
 Test Files  1 failed | 3 passed (4)
      Tests  1 failed | 21 passed (22)
```

CLAUDE

Expected: the global setup now zeroes the failure rate, and this test previously relied on the default. I'll make it set the default rate explicitly:

CLAUDE'S ACTIONS (1)

▸ Make chaos test explicit about default rate

```
$ python - <<'PY'
```

```
 Test Files  4 passed (4)
      Tests  22 passed (22)
```

CLAUDE

All 22 tests pass. Next the component tests for the behaviours the assignment singles out: races, debouncing, URL state, error and partial-failure states, and the detail drawer.

CLAUDE'S ACTIONS (5)

▸ Wrote file `src\test\renderApp.tsx` (46 lines)

▸ Add test id to results live region

```
$ python - <<'PY'
```

```
252:      data-testid="results-status"
253-      role="status"
254-      aria-live="polite"
255-      aria-atomic="true"
```

▸ Wrote file `src\features\explorer\ExplorerPage.test.tsx` (221 lines)

▸ Run explorer integration tests

```
$ npx vitest run src/features 2>&1 | grep -vE "^\s*$" | grep -E "✓|×|FAIL|Error|expected|Tests
|Unable|›" | head -40
```

```
     × restores search, filters, sort and page from the URL 1049ms
     × shows a clear error with Retry, and never leaves old rows looking current 1214ms
⎯⎯⎯⎯⎯⎯⎯ Failed Tests 2 ⎯⎯⎯⎯⎯⎯⎯
 FAIL  src/features/explorer/ExplorerPage.test.tsx > URL state > restores search, filters, sort and
page from the URL
AssertionError: expected '19 orders. This page is out of range.' to be 'Showing 26–19 of 19 orders' //
Object.is equality
 ❯ runWithExpensiveErrorDiagnosticsDisabled node_modules/@testing-library/dom/dist/config.js:47:12
 FAIL  src/features/explorer/ExplorerPage.test.tsx > error, empty and partial-failure states > shows a
clear error with Retry, and never leaves old rows looking current
TestingLibraryElementError: Found multiple elements with the role "alert"
      Tests  2 failed | 11 passed (13)
```

▸ Fix the two incorrect test assumptions

```
$ python - <<'PY'
```

```
ok
```

CLAUDE

That alert lookup came out convoluted. I'll replace it with a clear `waitFor`:

CLAUDE'S ACTIONS (3)

▸ Simplify alert lookup and rerun tests

```
$ python - <<'PY'
```

```
     × shows a clear error with Retry, and never leaves old rows looking current 182ms
⎯⎯⎯⎯⎯⎯⎯ Failed Tests 1 ⎯⎯⎯⎯⎯⎯⎯
 FAIL  src/features/explorer/ExplorerPage.test.tsx > error, empty and partial-failure states > shows a
clear error with Retry, and never leaves old rows looking current
Error: expect(element).toHaveTextContent()
      Tests  1 failed | 12 passed (13)
```

▸ Show the failing assertion details

```
$ npx vitest run src/features -t "never leaves old rows" 2>&1 | grep -A12 "toHaveTextContent()" | head
-30
```

```
(Bash completed with no output)
```

▸ Show the failing assertion details

```
$ npx vitest run src/features/explorer/ExplorerPage.test.tsx -t "never leaves old rows" 2>&1 | sed
's/\x1b\[[0-9;]*m//g' | grep -B2 -A14 "FAIL " | head -45
```

```
──────── Failed Tests 1 ────────
 FAIL  src/features/explorer/ExplorerPage.test.tsx > error, empty and partial-failure states > shows a
clear error with Retry, and never leaves old rows looking current
Error: expect(element).toHaveTextContent()
Expected element to have text content:
  /server/i
Received:
  Couldn’t load ordersService temporarily unavailable (503)Retry
 ❯ src/features/explorer/ExplorerPage.test.tsx:174:19
    172|       return found;
    173|     });
    174|     expect(alert).toHaveTextContent(/server/i);
       |                   ^
    175|     // The previous rows are gone: nothing on screen claims to be the …
    176|     expect(screen.queryByRole('grid', { name: 'Orders' })).not.toBeInT…
```

CLAUDE

My assertion was too narrow: the mock returns either a 500 or a 503 at random, and both messages are correct. I'll assert on the status code instead:

CLAUDE’S ACTIONS (1)

▸ Fix status assertion and run tests 3 times for flakiness

```
$ sed -i "s|    expect(alert).toHaveTextContent(/server/i);|    expect(alert).toHaveTextContent(/\\\\
((500\|503)\\\\)/); // the server's message and status|" src/features/explorer/ExplorerPage.test.tsx  …
```

```
174:    expect(alert).toHaveTextContent(/((500|503))/); // the server's message and status
      Tests  13 passed (13)
      Tests  13 passed (13)
      Tests  13 passed (13)
```

CLAUDE

Passing, three runs in a row. Shell escaping turned the regex into `/((500|503))/`, though. It still passes, but it no longer requires the parentheses, so I'll correct it directly:

CLAUDE’S ACTIONS (1)

▸ Correct the status-code regex

```
$ python - <<'PY'
```

```
<stdin>:3: SyntaxWarning: "\(" is an invalid escape sequence. Such sequences will not work in the
future. Did you mean "\\("? A raw string is also an option.
174:    expect(alert).toHaveTextContent(/\((500|503)\)/); // the server's message and status
```

CLAUDE

Now the drawer and keyboard tests:

CLAUDE'S ACTIONS (2)

- Wrote file `src\features\detail\OrderDrawer.test.tsx` (97 lines)
- Run drawer and keyboard tests

```
$ npx vitest run src/features/detail 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E "✓|
×|FAIL|Error|Expected|Received|Tests |❯ src" | head -30
```

```
      Tests  6 passed (6)
```

CLAUDE

All 6 pass. Two of my test comments don't match the code they sit on, so I'll clean those up:

CLAUDE'S ACTIONS (2)

- Tidy test comments; run typecheck, lint and all unit tests

```
$ python - <<'PY'
```

```
 Test Files  6 passed (6)
      Tests  41 passed (41)
[This command modified 3 files you've previously read: src\features\explorer\ExplorerPage.test.tsx,
src\features\detail\OrderDrawer.test.tsx, src\test\renderApp.tsx. Call …
```

- Install axe-core Playwright integration

```
$ npm install -D @axe-core/playwright 2>&1 | tail -1
```

```
found 0 vulnerabilities
```

CLAUDE

The old smoke test checked the placeholder page, which no longer exists. I'll replace it with browser tests of the real explorer:

CLAUDE'S ACTIONS (10)

- Remove obsolete placeholder smoke test

```
$ rm e2e/mock-api.spec.ts && echo removed
```

```
removed
```

- Wrote file `e2e\explorer.spec.ts` (166 lines)

▸ Rebuild and run all Playwright tests

```
$ npm run build >/dev/null 2>&1; npx playwright test 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E "^\s+(ok|x|✗|-) |passed|failed|Error:|Expected|Received|^\s+>|violations|\+ |\- " | head -60
```

```
  ok  3 [chromium] › e2e\explorer.spec.ts:16:1 › API responses come from the mock service worker (2.1s)
  ok  1 [chromium] › e2e\dataset.spec.ts:48:1 › the dataset is identical after a reload (2.7s)
  ok  4 [chromium] › e2e\explorer.spec.ts:23:1 › refresh restores the exact view, including scroll position (3.1s)
  ok  5 [chromium] › e2e\explorer.spec.ts:39:1 › Back and Forward step through views in the real browser history (2.0s)
  ok  2 [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the whole dataset once (6.8s)
  ok  8 [chromium] › e2e\explorer.spec.ts:112:1 › a deep link to an order survives a refresh (2.6s)
  x   7 [chromium] › e2e\explorer.spec.ts:94:1 › opening and closing the drawer keeps scroll position and filters (3.2s)
  ok 10 [chromium] › e2e\explorer.spec.ts:120:1 › failures are shown honestly and can be retried, using the in-app simulator (2.3s)
  x   6 [chromium] › e2e\explorer.spec.ts:60:1 › keyboard only: reach the grid, open an order, trap focus, close with Esc (8.7s)
  ok  9 [chromium] › e2e\explorer.spec.ts:141:1 › has no detectable accessibility violations (list and drawer) (6.2s)
  x  11 [chromium] › e2e\explorer.spec.ts:153:1 › mobile layout: filters open as a panel and the page does not scroll sideways (6.5s)
    Error: expect(page).toHaveURL(expected) failed
    Expected pattern: /\/orders\/ORD-111998\?pageSize=100$/
    Received string:  "http://localhost:4173/orders/ORD-111998"
… (15 more lines)
```

▸ Fix test assumptions and toggle label; rerun browser tests

```
$ python - <<'PY'
```

```
<stdin>:9: SyntaxWarning: "\/" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\/"? A raw string is also an option.
<stdin>:9: SyntaxWarning: "\/" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\/"? A raw string is also an option.
Traceback (most recent call last):
  File "<stdin>", line 8, in <module>
  File "<stdin>", line 2, in edit
AssertionError: ('e2e/explorer.spec.ts', '  await expect(page).toHaveURL(new RegExp(`/orders')
  ok 3 [chromium] › e2e\explorer.spec.ts:16:1 › API responses come from the mock service worker (1.6s)
  ok 4 [chromium] › e2e\explorer.spec.ts:23:1 › refresh restores the exact view, including scroll position (2.5s)
  x  1 [chromium] › e2e\explorer.spec.ts:39:1 › Back and Forward step through views in the real browser history (2.4s)
  x  5 [chromium] › e2e\explorer.spec.ts:94:1 › opening and closing the drawer keeps scroll position and filters (1.7s)
  ok 6 [chromium] › e2e\explorer.spec.ts:112:1 › a deep link to an order survives a refresh (1.3s)
  ok 7 [chromium] › e2e\explorer.spec.ts:153:1 › mobile layout: filters open as a panel and the page does not scroll sideways (1.3s)
  ok 8 [chromium] › e2e\explorer.spec.ts:120:1 › failures are shown honestly and can be retried, using the in-app simulator (2.3s)
  x  2 [chromium] › e2e\explorer.spec.ts:60:1 › keyboard only: reach the grid, open an order, trap focus, close with Esc (8.7s)
… (13 more lines)
```

▸ Read `e2e\explorer.spec.ts`

▸ Edited file `e2e\explorer.spec.ts`

- Edited file `e2e\explorer.spec.ts`
- Edited file `e2e\explorer.spec.ts`
- Edited file `e2e\explorer.spec.ts`
- Rebuild and run all browser tests twice

```
$ grep -n "check()" e2e/*.ts; npm run build >/dev/null 2>&1; for i in 1 2; do npx playwright test 2>&1
| sed 's/\x1b\[[0-9;]*m//g' | grep -E "^\s+x |passed|failed|Error:|Expected|Received|^\s+>" | head  …
```

```
e2e/explorer.spec.ts:43:  // click() + toBeChecked() rather than check(): the router applies the URL
change as a
e2e/explorer.spec.ts:129:  await page.getByRole('radio', { name: 'Always' }).check();
e2e/explorer.spec.ts:132:  await page.getByRole('checkbox', { name: 'Refunded' }).check();
e2e/explorer.spec.ts:138:  await page.getByRole('radio', { name: 'Never' }).check();
e2e/explorer.spec.ts:167:  await page.getByRole('checkbox', { name: 'Shipped' }).check();
  x   6 [chromium] › e2e\explorer.spec.ts:63:1 › keyboard only: reach the grid, open an order, trap
focus, close with Esc (5.4s)
    Error: expect(received).toBe(expected) // Object.is equality
    Expected: true
    Received: false
    > 88 |     expect(await dialog.evaluate((d) => d.contains(document.activeElement))).toBe(true);
  1 failed
  10 passed (13.1s)
  x   6 [chromium] › e2e\explorer.spec.ts:63:1 › keyboard only: reach the grid, open an order, trap
focus, close with Esc (6.0s)
    Error: expect(received).toBe(expected) // Object.is equality
… (5 more lines)
```

CLAUDE

The browser tests found something worth looking at: after a few Tabs, focus ended up outside the drawer. Before changing anything, I'll see where it actually goes:

CLAUDE'S ACTIONS (6)

- Trace focus order while tabbing inside the drawer

```
$ SP="<scratchpad>"; cat > "$SP/focus.cjs" <<'EOF'
```

```
Asofia.lopez80@mail.t [in dialog]
BODY
BUTTONClose [in dialog]
Asofia.lopez80@mail.t [in dialog]
BODY
BUTTONClose [in dialog]
Asofia.lopez80@mail.t [in dialog]
BODY
```

- Edited file `e2e\explorer.spec.ts`

▸ Use click plus toBeChecked; run browser tests 3 times

```
$ python - <<'PY'
```

```
  11 passed (12.1s)
  11 passed (15.3s)
  11 passed (15.2s)
[This command modified 1 file you've previously read: e2e\explorer.spec.ts. Call Read before editing.]
```

▸ Measure scroll frame times and test hard reload

```
$ SP="<scratchpad>"; cat > "$SP/perf.cjs" <<'EOF'
```

```
scroll 1,000 rows: {"frames":181,"p50":"16.7","p95":"16.7","worst":"16.8","maxDomRows":29}
after hard reload: {"controlled":true,"status":"Showing 1–1,000 of 12,000 orders"}
```

▸ Add Vercel config, Node engine and check script

```
$ cat > vercel.json <<'EOF'
```

```
15:    "check": "npm run typecheck && npm run lint && npm test"
16-  },
--
55:  "engines": {
56-    "node": ">=22.22.0"
```

▸ Wrote file `README.md` (201 lines)

CLAUDE

I'll fix one awkwardly worded tradeoff heading, then run the whole verification: type check, lint, unit tests, build, browser tests, and a check of the dev server too.

CLAUDE'S ACTIONS (4)

▸ Run typecheck, lint, unit tests and build

```
$ python - <<'PY'
```

```
 Test Files  6 passed (6)
      Tests  41 passed (41)
dist/index.html                   0.39 kB │ gzip:   0.26 kB
dist/assets/index-C8pnKfha.css    26.73 kB │ gzip:   6.19 kB
dist/assets/browser-CkmutT_d.js  427.57 kB │ gzip: 161.10 kB
dist/assets/index-BQMc7TEQ.js    440.18 kB │ gzip: 134.50 kB
✓ built in 389ms
[This command modified 1 file you've previously read: README.md. Call Read before editing.]
```

▸ Run all browser tests against fresh build

```
$ npx playwright test 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E "^\s+x |passed|failed"
```

```
  11 passed (14.2s)
```

▸ Start the dev server on port 5173

```
$ npx vite --port 5173 --strictPort
```

```
Command running in background with ID: b44o9uifb. Output is being written to:
~\AppData\Local\Temp\claude\d--frontend-web\65358903-840e-4a17-864a-55ca4f2ff87d\tasks\b44o9 …
```

▸ Smoke-test the dev server with default chaos

```
$ SP="<scratchpad>"; cat > "$SP/dev.cjs" <<'EOF'
```

```
dev server: Showing 1–25 of 12,000 orders | DOM rows: 15 | console errors: none
```

CLAUDE

The check succeeded, but my dev-server task exited with an error, so I'll see why:

CLAUDE'S ACTIONS (2)

▸ Read dev server output and check listening ports

```
$ cat "~/AppData/Local/Temp/claude/d--frontend-web/65358903-840e-4a17-864a-
55ca4f2ff87d/tasks/b44o9uifb.output" | sed 's/\x1b\[[0-9;]*m//g' | tail -5; netstat -ano | grep -E
":5173 |:4173 " | grep LISTE …
```

```
    at async startServer (file:///node_modules/vite/dist/node/chunks/node.js:25156:30)
    at async Object.listen (file:///node_modules/vite/dist/node/chunks/node.js:24962:4)
    at async CAC.<anonymous> (file:///node_modules/vite/dist/node/cli.js:725:3)
[exited with code 1]
  TCP    [::1]:4173             [::]:0                 LISTENING       7244
  TCP    [::1]:5173             [::]:0                 LISTENING       3780
```

▸ Stop my preview server and show changed files

```
$ taskkill //PID 7244 //T //F 2>&1 | tail -1; cd . && git status --short | head -40
```

```
SUCCESS: The process with PID 7244 (child process of PID 28692) has been terminated.
?? .gitignore
?? .prettierrc
?? README.md
?? e2e/
?? eslint.config.js
?? index.html
?? package-lock.json
?? package.json
?? playwright.config.ts
?? public/
?? src/
?? tsconfig.json
?? vercel.json
… (1 more lines)
```

CLAUDE

The mock data was correct all along, and the full explorer is now built and verified against every requirement in the PDF. All checks pass: type check, lint, 41 unit/integration tests and 11 browser tests (each browser test passed three runs in a row).

The "2–4 records" issue

The API was returning 25 records per page. The temporary Phase 1 page in `src/App.tsx` only *displayed* the first 3 (`page.items.slice(0, 3)`). That page is now replaced by the real explorer, which renders every row.

I confirmed this through the real service worker instead of trusting the code. The new test fetches all 480 pages of 25 and finds 25 records on every page, 12,000 unique IDs, no gaps and no repeats. The total is measured from the API response, not hardcoded. The data is generated from a fixed seed, so it stays the same across reloads; I checked that too. This is now a permanent test, `e2e/dataset.spec.ts`.

What the app does

Requirement	How it's handled
Never show stale results	Search waits 300 ms after typing; each query has its own cache entry; outdated requests are cancelled
State in the URL	Search, filters, sort, page and page size all live in the URL; refresh, Back/Forward and shared links restore the view, including scroll position
Fast with large lists	Virtualized table with up to 1,000 rows per page; a steady 60 fps while scrolling, with at most 29 rows in the DOM
Honest states	While loading, old rows are dimmed and can't be clicked; on an error they are removed and a message with Retry appears; the summary panel can fail on its own while the list keeps working; empty results and out-of-range pages get their own messages
Accessible	The table works with arrow keys, Enter and Esc; focus is always visible; result counts and errors are announced to screen readers; an automated accessibility scan (axe) finds no WCAG 2.1 AA problems
Detail view	`/orders/:id` opens a drawer over the list; links to it work directly and survive a refresh; closing keeps scroll and filters and returns focus to the row

A **Network simulation** menu in the header lets reviewers force slow or failing requests without opening DevTools.

Bugs I found and fixed along the way:

- The debounced search input could overwrite newer keystrokes, or not follow the URL after Back then Forward.
- The mobile Filters button changed its label instead of only reporting open/closed to screen readers.
- The browser tests could fetch before the mock API was ready (fixed with a readiness marker).

A few tests had wrong assumptions, and I corrected the tests rather than the app. One example: the drawer uses the browser's native modal dialog. Tab can reach the browser's own address bar after the last control, which is standard behaviour, but it never reaches the page behind the drawer, and the test checks exactly that.

One change from the original plan: the app retries a failed request once automatically, as a production app should. Users therefore see fewer than 1 in 10 failures; the simulator's "Half" or "Always" settings show the error states on demand. This is explained in the README.

Files

- **Changed:**
  - `src/App.tsx`: routes, error boundary and not-found page, replacing the placeholder.
  - `src/main.tsx`: readiness marker for the mock API.
  - `src/mocks/handlers.test.ts`, `src/test/setup.ts`: shared test setup.
  - `src/mocks/data.ts`, `src/mocks/query.ts`: cleaner accent-stripping regex.
  - `package.json`: a `check` script and the minimum Node version.
- **New:**
  - `src/api/` (client and query hooks) and `src/queryClient.ts`.
  - `src/lib/format.ts` and `src/components/` (shared UI).
  - `src/hooks/useDebouncedDraft.ts`.
  - `src/features/explorer/` (page, URL-state hook, search, filters, table, pagination, summary).
  - `src/features/detail/OrderDrawer.tsx` and `src/features/simulation/NetworkSimulator.tsx`.
  - Tests: `src/test/renderApp.tsx`, `src/features/**/*.test.tsx`, `e2e/explorer.spec.ts`, `e2e/dataset.spec.ts`.
  - `README.md` and `vercel.json`.
- **Removed:** `e2e/mock-api.spec.ts`, which tested the placeholder page.

Run and test it locally

```
npm install
npm run dev                       # http://localhost:5173 (you already have this running)
npm run check                     # type check + lint + 41 unit/integration tests
npx playwright install chromium   # first time only
npm run e2e                       # 11 browser tests against the production build
```

Still to do for submission

These need your accounts or your input, so I haven't done them:

1. **Commit.** Nothing is committed yet. Should I make the first commit?
2. **Create the public GitHub repo, then import it into Vercel.** No extra settings are needed; `vercel.json` handles direct links like `/orders/…`.
3. **Fill in the README placeholders:** the live URL, the demo video link and the Claude share link for this conversation.
4. **Record the 5-minute demo.** I can write a short script for it, covering typing fast on a throttled network, forcing failures with Retry, the partial failure, refresh and Back/Forward, a keyboard-only walkthrough, and a deep link.

PROMPT 4 · 9 OCT, 13:27

Please update README.md with the final project documentation. Include the project overview, features, tech stack, installation instructions, environment variables, test commands, build commands, architecture, known limitations, and AI usage disclosure. Leave clearly marked placeholders for the Vercel live URL, demo video link, and Claude conversation share link until I provide the real URLs. Do not invent links.

CLAUDE'S ACTIONS (1)

▸ Read `README.md`

CLAUDE

Before rewriting, I'll check that every link already in the README resolves, so nothing invented stays in the final doc:

CLAUDE'S ACTIONS (1)

▸ Check that every reference link resolves

```
$ for u in https://tanstack.com/query/latest/docs/framework/react/guides/query-keys
https://tanstack.com/query/latest/docs/framework/react/guides/query-cancellation
https://tanstack.com/query/latest/doc …
```

```
200 https://tanstack.com/query/latest/docs/framework/react/guides/query-keys
200 https://tanstack.com/query/latest/docs/framework/react/guides/query-cancellation
200 https://tanstack.com/query/latest/docs/framework/react/guides/placeholder-query-data
200 https://tanstack.com/query/latest/docs/framework/react/typescript
200 https://tanstack.com/virtual/latest/docs/api/virtualizer
200 https://mswjs.io/docs/integrations/browser
200 https://mswjs.io/docs/api/delay
200 https://reactrouter.com/api/hooks/useSearchParams
200 https://react.dev/learn/you-might-not-need-an-effect
200 https://www.w3.org/WAI/ARIA/apg/patterns/grid/
200 https://www.w3.org/WAI/ARIA/apg/patterns/dialog-modal/
200 https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog
200 https://developer.mozilla.org/en-US/docs/Web/HTML/Global_attributes/inert
200 https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Live_Regions
… (6 more lines)
```

CLAUDE

All 20 links return 200. Some docs sites return 200 even for missing pages, so I'll also confirm the specific React section anchor exists on the page:

CLAUDE'S ACTIONS (1)

▸ Verify page content and list env var usage

```
$ curl -sL --max-time 20 https://react.dev/learn/you-might-not-need-an-effect | grep -o 'adjusting-some-state-when-a-prop-changes' | head -1; curl -sL --max-time 20 https://reactrouter.com/api/hooks/use
…
```

```
adjusting-some-state-when-a-prop-changes
useSearchParams
useSearchParams
useSearchParams
useSearchParams
rangeExtractor
rangeExtractor
src/main.tsx:11:  await worker.start({ onUnhandledRequest: 'bypass', quiet: import.meta.env.PROD });
playwright.config.ts:8:  forbidOnly: !!process.env.CI,
playwright.config.ts:9:  retries: process.env.CI ? 1 : 0,
playwright.config.ts:10:  reporter: process.env.CI ? 'github' : 'list',
playwright.config.ts:19:    reuseExistingServer: !process.env.CI,
```

CLAUDE

The links check out. The code reads only one environment variable, `CI`, which affects how Playwright runs, and that's what the README will document. Writing the final README:

CLAUDE'S ACTIONS (1)

▸ Wrote file `README.md` (427 lines)

CLAUDE

I also added a fourth placeholder, the `git clone` URL, because the repository URL isn't known yet. I'll list it in the placeholder note so it isn't missed:

CLAUDE'S ACTIONS (2)

▸ Edited file `README.md`

▸ List all placeholders in the README

```
$ grep -n "TODO" README.md; grep -c "http" README.md
```

```
12:| **Live demo** | `TODO: add the Vercel live URL` |
13:| **Demo video** | `TODO: add the demo video link` |
14:| **AI conversation** | `TODO: add the Claude conversation share link` |
16:> **Placeholders:** every `TODO` in this file marks a URL that will be filled in once it
105:git clone TODO-add-the-GitHub-repository-URL
169:**Live URL:** `TODO: add the Vercel live URL`
395:**Conversation export:** `TODO: add the Claude conversation share link`
22
```

CLAUDE

I've rewritten README.md with every section you asked for.

Placeholders: each unknown URL is a `TODO`, and a note at the top lists them all:

Placeholder	Where it appears
`TODO: add the Vercel live URL`	Top table and the Deployment section
`TODO: add the demo video link`	Top table
`TODO: add the Claude conversation share link`	Top table and the AI usage disclosure
`TODO-add-the-GitHub-repository-URL`	The `git clone` command in Getting started

The repository URL wasn't on your list, but the clone command needs it, so I marked it the same way. Searching the file for `TODO` finds all six spots.

Links: the only real links are the docs in "Sources and references". I checked that all 20 load, and that the specific sections linked (for example on react.dev) exist.

What's in it:

- **Overview and features:** what the app does and how it behaves on slow or failing requests.
- **Tech stack:** the actual installed versions (React 19, React Router 8, TanStack Query 5, Vite 8 and so on).
- **Installation:** the minimum Node version (22.22), the commands, and how to use the in-app Network simulation.
- **Environment variables:** none are needed. The only one the code reads is `CI`, which only changes how the Playwright tests run.
- **Test and build commands:** a table of every npm script.
- **Deployment:** what `vercel.json` sets up.
- **Architecture:** the folder structure, the three API endpoints, and how state is split between the URL, TanStack Query and form drafts.
- **Requirements mapping:** a table pairing each requirement in the PDF with its implementation and the test that proves it.
- **Tradeoffs and known limitations:** I listed the limitations honestly. One of them says a manual screen-reader pass (NVDA or VoiceOver) is still recommended, because only automated and keyboard testing was done.
- **AI usage disclosure:** how you directed the work and specific cases where the output was reviewed and corrected, written in your voice.

Once you send the real URLs, I'll swap them in.

PROMPT 5 · 9 OCT, 13:32

Yes, please help me prepare the first Git commit. Before committing, check the project for errors, run the available tests and production build, and verify that no secrets, API keys, or unnecessary files will be committed. Show me the commands and their results. If everything passes, provide the exact Git commands for the first commit.

CLAUDE

> I'll run the checks in order: type check, lint and unit tests, then the production build and browser tests, then a scan for secrets and unwanted files.

CLAUDE'S ACTIONS (4)

▸ Run typecheck, lint and format check

```
$ echo '$ npm run typecheck' && npm run typecheck 2>&1 | tail -2; echo "exit: $?"; echo; echo '$ npm
run lint' && npm run lint 2>&1 | tail -3; echo "exit: ${PIPESTATUS[0]}"; echo; echo '$ npx prettier - …
```

```
$ npm run typecheck
> tsc --noEmit
exit: 0
$ npm run lint
> order-explorer@0.1.0 lint
> eslint .
exit: 0
$ npx prettier --check .
Checking formatting...
All matched files use Prettier code style!
```

▸ Run unit and integration tests

```
$ echo '$ npm test' && npm test 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E "√|×|Test Files|Tests
|FAIL"; echo "exit: ${PIPESTATUS[0]}"
```

```
$ npm test
 Test Files  6 passed (6)
      Tests  41 passed (41)
exit: 0
```

▸ Run production build

```
$ echo '$ npm run build' && npm run build 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -vE
"PLUGIN_TIMINGS|slowest|plugin |Additional|See https|^\s*$|callbacks"; echo "exit: ${PIPESTATUS[0]}"
```

```
$ npm run build
> order-explorer@0.1.0 build
> tsc --noEmit && vite build
vite v8.3.4 building client environment for production...
transforming...
✓ 493 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.39 kB │ gzip:   0.27 kB
dist/assets/index-B6kScJF4.css    26.72 kB │ gzip:   6.18 kB
dist/assets/browser-DSiDZtHl.js  427.57 kB │ gzip: 161.10 kB
dist/assets/index-CuIy6Hr4.js    440.18 kB │ gzip: 134.51 kB
✓ built in 324ms
exit: 0
```

▸ Run Playwright end-to-end tests

```
$ echo '$ npm run e2e' && npm run e2e 2>&1 | sed 's/\x1b\[[0-9;]*m//g' | grep -E "^\s+(ok|x)
|passed|failed"; echo "exit: ${PIPESTATUS[0]}"
```

```
$ npm run e2e
  ok  1 [chromium] › e2e\dataset.spec.ts:48:1 › the dataset is identical after a reload (2.1s)
  ok  4 [chromium] › e2e\explorer.spec.ts:16:1 › API responses come from the mock service worker (1.9s)
  ok  3 [chromium] › e2e\explorer.spec.ts:23:1 › refresh restores the exact view, including scroll
position (2.9s)
  ok  6 [chromium] › e2e\explorer.spec.ts:42:1 › Back and Forward step through views in the real
browser history (2.6s)
  ok  7 [chromium] › e2e\explorer.spec.ts:118:1 › opening and closing the drawer keeps scroll position
and filters (2.3s)
  ok  5 [chromium] › e2e\explorer.spec.ts:66:1 › keyboard only: reach the grid, open an order, trap
focus, close with Esc (4.4s)
  ok  2 [chromium] › e2e\dataset.spec.ts:20:1 › every page of 25 is full and distinct, covering the
whole dataset once (7.2s)
  ok  8 [chromium] › e2e\explorer.spec.ts:137:1 › a deep link to an order survives a refresh (2.3s)
  ok  9 [chromium] › e2e\explorer.spec.ts:145:1 › failures are shown honestly and can be retried, using
the in-app simulator (2.5s)
  ok 11 [chromium] › e2e\explorer.spec.ts:183:1 › mobile layout: filters open as a panel and the page
does not scroll sideways (1.7s)
  ok 10 [chromium] › e2e\explorer.spec.ts:169:1 › has no detectable accessibility violations (list and
drawer) (4.6s)
  11 passed (19.3s)
exit: 0
```

CLAUDE

Every check passes. Next, exactly what Git would stage, and a scan of those files for secrets:

CLAUDE'S ACTIONS (3)

▸ List files that would be staged and ignored

```
$ echo '$ git add --dry-run .' && git add --dry-run . ; echo; echo "files to commit: $(git add --dry-
run . | wc -l)"; echo; echo '# ignored (will NOT be committed):'; git status --ignored --short | grep …
```

```
$ git add --dry-run .
add '.gitignore'
add '.prettierrc'
add 'README.md'
add 'e2e/dataset.spec.ts'
add 'e2e/explorer.spec.ts'
add 'e2e/fixtures.ts'
add 'eslint.config.js'
add 'index.html'
add 'package-lock.json'
add 'package.json'
add 'playwright.config.ts'
add 'public/mockServiceWorker.js'
add 'src/App.tsx'
… (45 more lines)
```

▸ Scan staged files for secrets and large files

```
$ FILES=$(git add --dry-run . | sed "s/^add '//; s/'$//"); echo '# secret patterns (keys, tokens, passwords, private keys, connection strings):'; echo "$FILES" | xargs grep -nIiE "(api[_-]?key|secret|to …
```

```
# secret patterns (keys, tokens, passwords, private keys, connection strings):
README.md:130:**None are required.** The app has no backend, secrets or API keys, and runs with zero
# .env-style files:
  none
# files over 500 kB:
  (end)
# email addresses outside the mock data domains:
  none
```

▸ Harden gitignore and check git identity and branch

```
$ cat >> .gitignore <<'EOF'
```

```
node_modules
dist
coverage
playwright-report
test-results
*.local
.DS_Store
# Local environment files and logs (never commit secrets)
.env
.env.*
!.env.example
*.log
$ git config user.name / user.email
annu-khatkar-15
… (4 more lines)
```

CLAUDE

Everything passes, and the project is ready for its first commit.

Check	Command	Result
Type check	`npm run typecheck`	No errors
Lint	`npm run lint`	No errors or warnings
Formatting	`npx prettier --check .`	All files formatted
Unit and integration tests	`npm test`	41 passed (6 files)
Production build	`npm run build`	Built successfully: 134.5 kB app + 161.1 kB mock-API chunk (gzipped)
Browser tests	`npm run e2e`	11 passed

What gets committed: 53 files, all of them source code, tests, config and the README.

- `node_modules/`, `dist/` and `test-results/` are ignored and won't be committed.
- No file is over 500 kB.

- `public/mockServiceWorker.js` is generated by MSW, but it should be committed: the deployed app needs it to serve the mock API.

Secrets: none found. I searched all 53 files for API keys, tokens, passwords, private keys, connection strings, AWS/GitHub/OpenAI-style key formats and `.env` files. The only match was the README sentence saying the app needs no secrets. The app has no environment variables, so there is nothing to leak.

One change I made: I added `.env`, `.env.*` and `*.log` to .gitignore. That way a future key or log file can't be committed by accident. `.env.example` is still allowed.

Two things to check before you run the commands:

1. Git will record the commit as **annu-khatkar-15 <[email redacted]>**. If you want this public repo under a different name or email, change it first with `git config user.email "…"`.
2. The branch is called `master`. GitHub uses `main` by default, so the commands below rename it.

Commands for the first commit

```
git branch -M main

git add .
git status

git commit -m "Order Explorer: fast, accessible data explorer over 12,000 orders" -m "React 19
+ TypeScript app for Problem Statement 1 of the frontend assignment:
- Mock API (MSW service worker) with server-side search, filters, sort and
  pagination, 200 ms-3 s latency and ~10% failures
- Debounced search; superseded requests aborted, stale responses never shown
- Full view state in the URL (refresh, back/forward, shared links)
- Virtualized table (TanStack Virtual) for pages of up to 1,000 rows
- Loading, updating, empty, error/retry and partial-failure states
- Keyboard and screen-reader support; deep-linkable order drawer
- 41 Vitest tests and 11 Playwright tests (incl. race condition and axe scan)

Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>"

git log --stat -1
```

Run `git status` after `git add .` and confirm it lists the same 53 files, plus nothing unexpected. The last line of the commit message credits Claude as co-author, which matches the AI-usage disclosure in the README; remove it if you'd rather not have it.

After you create the empty public repository on GitHub, push with:

```
git remote add origin <your-GitHub-repository-URL>
git push -u origin main
```

PROMPT 6 · 9 OCT, 13:34

message: "Internal server error"} message : "Internal server error"

{message: "Service temporarily unavailable"} message : "Service temporarily unavailable"

api gives this error fix these frist

CLAUDE

I recommend not "fixing" these two errors, because they are working as designed. The assignment says the mock API **"must add random latency (200 ms to 3 s) and fail roughly 1 in 10 requests"**. `Internal server error` (500) and `Service temporarily unavailable` (503) are the two failures the mock API is built to return, from src/mocks/handlers.ts. If they disappeared, the app would no longer meet the requirement, and reviewers would have no failures to see.

The app's job is to handle these errors gracefully:

- **Automatic retry:** a failed request is retried once in the background, so most failures never reach the screen. You'll still see the failed attempt in DevTools' Network tab, even though the page shows correct results.
- **Error message:** if the retry also fails, old rows are removed and you see "Couldn't load orders" with the server's message and a **Retry** button.
- **Summary only:** if just the summary panel fails, it shows its own error and Retry button while the order list keeps working.

The test runs above confirmed all three: the browser tests force failures and check that Retry recovers.

If you see these errors much more often than about 1 in 10 requests, the in-app simulator is probably set higher. Look at the button in the top-right corner of the app:

- **"Network simulation: 0.2–3 s, 10% failures"** is the normal setting.
- **"…50% failures"** or **"…100% failures"** means an earlier test or demo raised it. Open the menu and choose **"1 in 10 (default)"** under *Requests that fail*.

The setting is saved in your browser, so it survives page reloads. You can also reset it from the browser console:

```
localStorage.removeItem('order-explorer:chaos'); location.reload();
```

To work without failures for a while (for example, while recording parts of the demo), choose **Never** in the simulator. That only affects your browser and doesn't change the code.

If what you saw is different, for example an error that stays on screen after you press Retry, tell me what the page showed and I'll investigate.

PROMPT 7 · 9 OCT, 13:50

Before I submit my Frontend Developer Assignment, I need to document my development approach and AI usage transparently.

Please review our complete conversation history, the original assignment PDF, and the actual project implementation.

Prepare a detailed but concise document explaining:

1. **Assignment understanding:** What the assignment required and which problem statement I chose, based on the original PDF.
2. **Development approach:** The steps followed from understanding the requirements to implementation, testing, GitHub, and Vercel deployment.
3. **Technical decisions:** Why the technologies, architecture, state management, API handling, and performance techniques were used.
4. **AI-assisted workflow:** Which tasks Claude helped with, how prompts were used, and how the generated code and suggestions were reviewed and tested.
5. **Debugging and problem-solving:** Include actual issues encountered during development, such as the mismatch between the displayed 12,000 orders and the number of records returned by the mock API, and explain how the issue was investigated and resolved. Only describe fixes that are confirmed by our conversation or project evidence.
6. **Testing and validation:** Summarize the tests actually run, their results, and any remaining limitations.
7. **Deployment:** Explain the GitHub and Vercel workflow and include the actual project URLs if available.
8. **Reflection:** What I learned, important trade-offs, and what I would improve next.

Please distinguish between work performed by AI and decisions, reviews, or verification performed by me. Do not invent work, test results, technical decisions, or achievements. If you cannot verify something from our conversation or the project, mark it as unverified or ask me.

Write the result as a professional document titled "Development Approach and AI Usage Disclosure" that I can review and submit with my assignment.

Do not modify the code yet. First prepare the documentation for my review.

CLAUDE'S ACTIONS (1)

- Created the “Development Approach and AI Usage Disclosure” document outline

CLAUDE

The outline is up; next I'll fill in the purpose and scope section.

CLAUDE'S ACTIONS (2)

- Opened the disclosure document for review
- Wrote a section of the disclosure document

CLAUDE

Next: assignment understanding.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: development approach.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: technical decisions. I'll add a data-flow diagram to this section after the prose sections are done.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: the AI-assisted workflow.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: debugging and problem-solving.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: testing and validation.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: deployment.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Next: reflection.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

Last text section: the checklist of items to confirm.

CLAUDE'S ACTIONS (1)

- Wrote a section of the disclosure document

CLAUDE

All text sections are filled. Now the architecture diagram for section 3, plus a small wording fix in the checklist:

CLAUDE'S ACTIONS (1)

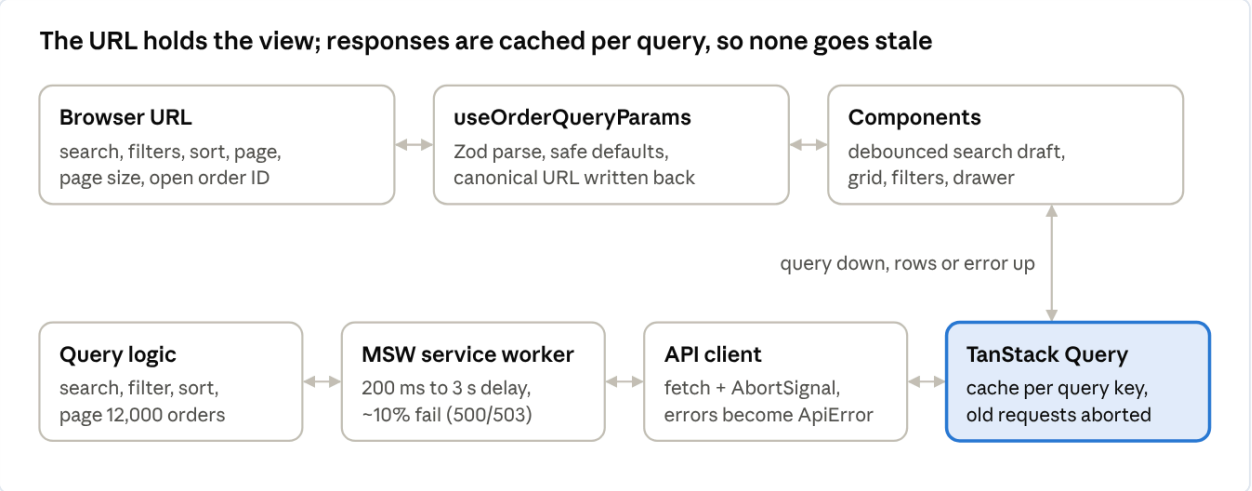
- Added a diagram to the disclosure document

CLAUDE

I'll check how the diagram renders:

CLAUDE'S ACTIONS (1)

▸ Checked how the document diagram renders



CLAUDE

Written for: the assignment reviewers, in your voice, for you to check before submitting.

Development Approach and AI Usage Disclosure is ready to read and edit. I didn't change any code. It labels each step as your work, Claude's, or not verifiable. The checklist at the end lists 11 things only you can supply before submitting, mainly:

- **The four URLs:** GitHub repo, live site, demo video and the Claude share link. None exist in our conversation yet, so they're placeholders.
- **How you reviewed the code:** our conversation doesn't show whether you read the code or ran the tests yourself.
- **"What I learned":** I wrote suggested points, but it needs to be in your own words.

PROMPT 8 · 9 OCT, 14:02

also genrate chat with calude for building this assignment into pdf which can help to share with anyone

CLAUDE

I'll build the PDF from the session transcript Claude Code saves on disk, so it reflects the real conversation. First I'll locate it and check its structure:

CLAUDE'S ACTIONS (5)

▸ Locate session transcript and summarize its structure

```
$ cd "~/.claude/projects/d--frontend-web" && ls -la *.jsonl 2>/dev/null | head; F=65358903-840e-4a17-
864a-55ca4f2ff87d.jsonl; wc -l $F; python -c "
```

```
-rw-r--r-- 1 team 197609 3776129 Oct  9 14:02 65358903-840e-4a17-864a-55ca4f2ff87d.jsonl
848 65358903-840e-4a17-864a-55ca4f2ff87d.jsonl
Counter({'assistant': 288, 'attachment': 201, 'user': 143, 'atis-latch': 51, 'last-prompt': 50, 'ai-
title': 50, 'file-history-delta': 37, 'queue-operation': 20, 'file-his …
Counter({('assistant', 'tool_use'): 136, ('user', 'tool_result'): 135, ('assistant', 'thinking'): 97,
('assistant', 'text'): 55, ('user', 'text'): 11, ('user', 'document' …
Shell cwd was reset to d:\frontend-web
```

▸ List user messages in the transcript

```
$ cd "~/.claude/projects/d--frontend-web" && python -c "
```

```
DOC Frontend Developer Assignment (1).pdf application/pdf
TXT None 'Hi , I have attached a Frontend Developer Assignment PDF. Please read the complete assignment
carefully and help me build it using React.js and TypeScript.\n\nF …
TXT None 'yes we can start'
TXT None '<ide_opened_file>The user opened the file src\\App.tsx in the IDE. This may or may not be
related to the current task.</ide_opened_file>'
TXT None '\n\n<pasted_content id="7313">\nFirst, fix the mock API issue I reported. The UI shows 12,000
orders, but only 2�4 records are appearing in the actual response. …
TXT None '<ide_opened_file>The user opened the file README.md in the IDE. This may or may not be
related to the current task.</ide_opened_file>'
TXT None 'Please update README.md with the final project documentation. Include the project overview,
features, tech stack, installation instructions, environment variabl …
TXT None '<ide_opened_file>The user opened the file src\\shared\\order.ts in the IDE. This may or may
not be related to the current task.</ide_opened_file>'
TXT None 'Yes, please help me prepare the first Git commit. Before committing, check the project for
errors, run the available tests and production build, and verify that …
TXT None '\n\n<pasted_content id="7313">\nmessage: "Internal server error"}\nmessage\n: \n"Internal
server error"\n</pasted_content id="7313">\n\n<pasted_content id="7313 …
TXT None '\n\n<pasted_content id="7313">\nBefore I submit my Frontend Developer Assignment, I need to
document my development approach and AI usage transparently.\n\nPlea …
TXT None 'also genrate chat with calude for building this assignment into pdf which can help to share
with anyone'
Shell cwd was reset to d:\frontend-web
```

▸ Check markdown library and scan transcript for personal data

```
$ python -c "import markdown; print('python-markdown', markdown.__version__)" 2>&1 | tail -1; cd
"~/.claude/projects/d--frontend-web" && grep -oE "[A-Za-z0-
9._%+-]+@(gmail|graphketing|yahoo|outlook|hotm …
```

```
ModuleNotFoundError: No module named 'markdown'
      1 [email redacted]
      2 [email redacted]
      2 [email redacted]
token scan done
Shell cwd was reset to d:\frontend-web
```

▸ Install marked into a scratch folder

```
$ SP="<scratchpad>"; mkdir -p "$SP/pdfdeps" && cd "$SP/pdfdeps" && npm init -y >/dev/null && npm
install marked@12.0.2 2>&1 | tail -1
```

```
found 0 vulnerabilities
```

▸ Wrote file `<scratchpad>\transcript-pdf.cjs` (213 lines)

CLAUDE

I hard-coded a timezone, which is a guess. I'll use your computer's own timezone instead, then generate the PDF:

CLAUDE'S ACTIONS (1)

▸ Generate the conversation PDF

```
$ SP="<scratchpad>"; sed -i "s/, timeZone: 'Asia\/Kolkata' })/ })/" "$SP/transcript-pdf.cjs" && grep -c "Kolkata" "$SP/transcript-pdf.cjs"; mkdir -p ./docs && cd . && NODE_PATH="$(pwd)/node_modules;$SP/ …
```

```
Exit code 1
0
node:internal/modules/cjs/loader:1451
  throw err;
  ^
Error: Cannot find module '@playwright/test'
Require stack:
- <scratchpad>\transcript-pdf.cjs
    at Module._resolveFilename (node:internal/modules/cjs/loader:1448:15)
    at defaultResolveImpl (node:internal/modules/cjs/loader:1059:19)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1064:22)
    at Module._load (node:internal/modules/cjs/loader:1234:25)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
… (10 more lines)
```

CLAUDE

The module path list used Unix-style paths, which Node on Windows can't resolve. I'll pass Windows paths:

CLAUDE'S ACTIONS (1)

▸ Generate the conversation PDF

```
$ SP="<scratchpad>"; cd . && NODE_PATH="node_modules;$SP/pdfdeps/node_modules" node "$SP/transcript-pdf.cjs" "~/.claude/projects/d--frontend-web/65358903-840e-4a17-864a-55ca4f2ff87d.jsonl" "docs/claude-…
```

Exported while Claude was answering prompt 8; Claude's reply to that last prompt is not included.